

02

Štampanje i rad sa tekstom

Biološki podaci su tekst: DNK i proteinske sekvence u programu čuvamo kao stringove, nizove karaktera

Zašto počinjemo tekstom, a ne brojevima?

A

T

G

C

G

T

A

String je niz karaktera. U kodu kažemo **string**, a za DNK i proteine **sekvenca**.

Sekvence

DNK i proteinske sekvence su nizovi slova, dakle tekst.

Parsiranje

U lancu alata (pipeline) naš program čita izlaz drugih programa.

Izlaz za druge

I pravi izlaz u formatu koji sledeći program razume.

Prva naredba: print

```
print("Hello world")
```

Naredba

Cela linija je jedna naredba (statement).

Funkcija: print

Kaže šta radimo. Iza imena uvek idu zagrade.

Argument

Kaže tačno šta: tekst koji se ispisuje.

Hello world

IZLAZ

„Štampanje” u programiranju znači ispis teksta na ekran, u terminal, a ne štampanje na papiru. Više argumenata se razdvaja zarezima.

Navodnici su bitni

```
print("Hello world")
print('Hello world')
# navodnici unutar stringa
print("She said, 'Hello world'")
print('He said, "Hello world"')
# ili obrnuta kosa crta (escape)
print('It\'s a gene')
```

IZLAZ

```
Hello world
Hello world

She said, 'Hello world'
He said, "Hello world"

It's a gene
```

Navodnici nisu deo stringa: u izlazu ih nema.

Navodnici odvajaju podatke od instrukcija, npr. od imena funkcije.

Jednostruki i dvostruki navodnici su potpuno ravnopravni.

Početni i krajnji navodnik moraju da se slože.

Komentari i poruke o greškama

Komentari

```
# odšampaj prijateljski pozdrav  
print("Hello world")
```

Sve od znaka `#` do kraja linije Python ignoriše. Komentar je dokumentacija koja se ne odvaja od koda i podsetnik za vas same kad se vratite kodu posle vikenda.

Tri česte greške

```
print(Hello world)
```

`SyntaxError: invalid syntax`
zaboravljeni navodnici

```
prin("Hello world")
```

`NameError: name 'prin' is not defined`
pogrešno napisano ime funkcije

```
print("Hello  
World")
```

`SyntaxError: unterminated string literal`
string ne sme da pređe u novi red

Poruka kaže fajl, liniju i tip greške. Procena tačnog mesta (znak `^`) ume da promaši, pa se na nju ne oslanjajte previše.

Kako se čita poruka o grešci

```
1 $ python error.py
2 Traceback (most recent call last):
3   File "error.py", line 1, in <module>
4     prin("Hello world")
5 NameError: name 'prin' is not defined
```

- 1 komanda kojom smo pokrenuli program
- 2 početak izveštaja o grešci
- 3 fajl i broj linije u kojoj je greška
- 4 sama linija koda
- 5 tip greške i opis: ime `prin` ne postoji

Kod greške `SyntaxError` Python ispod linije stavlja znak `^` tamo gde je primetio problem. Ta procena ume dosta da promaši, pa se na nju ne oslanjajte previše.

Programi skoro nikad ne rade ispravno iz prvog pokušaja.

Python ne pogađa šta ste mislili: zaustavlja se i prijavljuje grešku. Zato je čitanje poruka o greškama osnovna veština.

Specijalni karakteri i promenljive

Specijalni karakteri

`\n` novi red
`\t` tab
`\r` povratak na početak reda, bez novog reda

```
print("Hello\nworld")
```

```
Hello  
world
```

IZLAZ

Promenljive

```
# DNK sekvenca u promenljivoj my_dna  
my_dna = "ATGCGTA"  
print(my_dna)  
# nova vrednost iste promenljive  
my_dna = "TGGTCCA"
```

- Ime je proizvoljno, ali neka opisuje sadržaj: `my_dna` je dobro ime, `banana` loše
- Samo slova, cifre i `_`, a ime ne počinje cifrom
- Ne koristite ugrađene reči, npr. `print`
- Velika i mala slova se razlikuju: `my_dna` i `MY_DNA` su dve promenljive

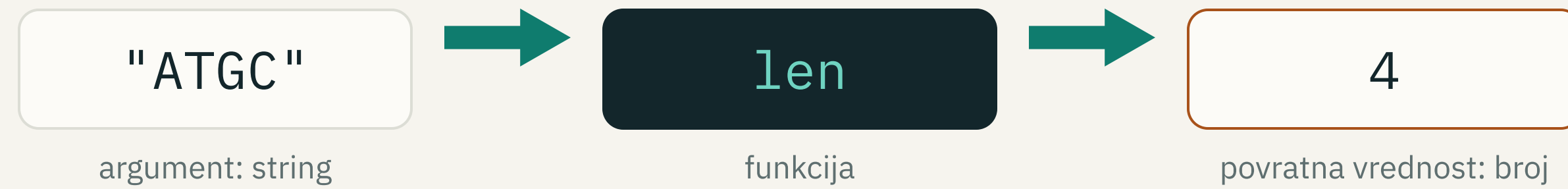
Imena promenljivih su proizvoljna

```
# radi isto i sa imenom banana
banana = "ATGCGTA"
print(banana)
# ime ne mora da odgovara vrednosti
my_file = "my_file.txt"
```

Računaru je ime svejedno, ljudima nije: `my_dna` kaže šta je unutra, `banana` ne kaže ništa. Ime promenljive i string su dve odvojene stvari.

IME	MOŽE?
my_dna, dna_2	da
2dna	ne: počinje cifrom
my-dna, dna%	ne: dozvoljena su samo slova, cifre i _
for, if, def	ne: ključne reči jezika
print	tehnički da, ali onda print više ne radi
MY_DNA	da, ali je to druga promenljiva od my_dna

Povratna vrednost: len



```
# ova linija ne ispisuje ništa  
len("ATGC")  
# vrednost sačuvamo u promenljivoj...  
dna_length = len("AGTC")  
# ...pa je iskoristimo  
print(dna_length)
```

`print` ispisuje tekst na ekran, a `len` vraća vrednost koju možemo da sačuvamo i dalje koristimo.

Rezultat `len` je broj, a ne string. To će biti važno na sledećem slajdu.

Stringovi i brojevi su različiti tipovi

```
my_dna = "ATGCGAGT"  
# len vraća broj (int), a ne string  
dna_length = len(my_dna)  
print("The length of the DNA sequence is " + dna_length)  
# ispravno: str pretvara broj u string  
print("The length of the DNA sequence is " + str(dna_length))
```

IZLAZ ČETVRTE LINIJE

TypeError: can only concatenate str (not "int") to str

Python ne zna kako da spoji string (**str**) i ceo broj (**int**).

Spajanje stringova: +

"AATT" + "GGCC" daje "AATTGGCC". Rezultat je opet string, pa spajanje može i unutar **print**.

Funkcija unutar funkcije

```
my_dna = "ATGCGAGT"  
print("length: " + str(len(my_dna)))
```

#	IZRAZ	VREDNOST
1	len(my_dna)	8
2	str(8)	"8"
3	"length: " + "8"	"length: 8"
4	print("length: 8")	ispis na ekran

str je funkcija koja prima jedan *argument* tipa *broj* i vraća *vrednost* tipa *string* koja predstavlja taj broj.

Računa se iznutra ka spolja. Svaka otvorena zagrada mora da se zatvori: ovde ih je tri na kraju.

Metode: alati koji pripadaju tipu

Funkcija

```
len(my_dna)
```

Ime funkcije, pa argumenti u zagradama.

Metod

```
my_dna.lower()
```

Promenljiva, tačka, ime metoda. Radi samo na svom tipu.

Vraća kopiju

```
my_dna.upper()
```

Original ostaje isti: rezultat sačuvajte ili ispišite.

```
protein = "vlspadktnv"  
# zameni valin tirozinom  
print(protein.replace("v", "y"))  
print(protein.replace("vls", "ymt"))  
# original se ne menja  
print(protein)
```

IZLAZ

```
ylspadktny  
ymtpadktnv  
  
vlspadktnv
```

lower i upper ne menjaju original

```
my_dna = "ATGC"  
print("before: " + my_dna)  
lowercase_dna = my_dna.lower()  
print("after: " + my_dna)  
print("lower: " + lowercase_dna)
```

IZLAZ

```
before: ATGC  
  
after: ATGC  
lower: atgc
```

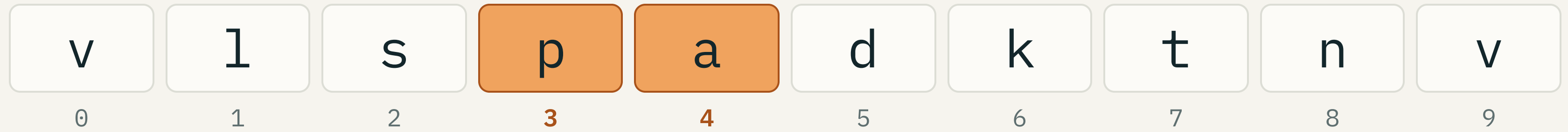
Metod pripada tipu

```
my_number = len("AGTC")  
print(my_number.lower())
```

AttributeError: 'int' object has no attribute 'lower'

Broj (int) nema metod lower. Metodi rade samo na tipu kojem pripadaju.

Izdvajanje dela stringa



```
protein = "vlspadktnv"  
print(protein[3:5])  
print(protein[0:6])  
print(protein[0:60])  
print(protein[0])
```

IZLAZ

```
pa  
vlspad  
vlspadktnv  
v
```

- Brojanje počinje od 0: `protein[3]` je četvrti karakter
- Početak je uključen, kraj nije: `[3:5]` daje pozicije 3 i 4
- Kraj iza kraja stringa znači „do kraja”
- Jedan broj daje jedan karakter

Brojanje i traženje podstringova

```
protein = "vlspadktnv"  
print(protein.count('v'))  
print(protein.count('lsp'))  
print(protein.count('w'))  
print(protein.find('p'))  
print(protein.find('kt'))  
print(protein.find('w'))
```

IZLAZ

```
2  
1  
0  
3  
6  
-1
```

- `count` vraća broj pojavljivanja (int)
- `find` vraća indeks prvog pojavljivanja, a `-1` ako podstringa nema
- Redosled je bitan: `my_dna.count(my_motif)` nije isto što i `my_motif.count(my_dna)`

Oba traže samo tačne podstringove. Za promenljive motive, kao što su vezivna mesta transkripcionih faktora, trebaju nam regularni izrazi (poglavlje 7).

Korak 1: proverite brojeve pre računanja

```
my_dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATCGATGCGTTTCAT"  
length = len(my_dna)  
a_count = my_dna.count('A')  
t_count = my_dna.count('T')  
print("length: " + str(length))  
print("A count: " + str(a_count))  
print("T count: " + str(t_count))
```

IZLAZ

```
length: 54  
A count: 16  
T count: 21
```

Izgleda razumno, ali da li je tačno? Ručno brojanje je sporo, a šta ako sekvenca ima 51 kilobazu? Bolje je pokrenuti isti kod na kratkoj sekvenci "ATGC": dužina 4, jedno A, jedno T.

Korak 2: formula (A + T) / dužina

```
my_dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATCGATGCGTTTCAT"  
length = len(my_dna)  
a_count = my_dna.count('A')  
t_count = my_dna.count('T')  
at_content = (a_count + t_count) / length  
print("AT content is " + str(at_content))
```

IZLAZ

```
AT content is 0.6851851851851852
```

Prvo test na maloj sekvenci

Za "ATGC" odgovor znamo napamet: 0.5. Bez zagrada, `a_count + t_count / length` daje 1.25, jer deljenje ima prednost nad sabiranjem.

Komplement DNK: zamena u četiri koraka

Naivno, i pogrešno

```
replacement1 = my_dna.replace('A', 'T')
replacement2 = replacement1.replace('T', 'A')
replacement3 = replacement2.replace('C', 'G')
replacement4 = replacement3.replace('G', 'C')
print(replacement4)
```

ACACAACCAAAACCAAAACA...

IZLAZ

Druga zamena vraća u A i one T koji su upravo nastali od A.
Ostaju samo A i C.

Trik: privremeno mala slova

```
replacement1 = my_dna.replace('A', 't')
replacement2 = replacement1.replace('T', 'a')
replacement3 = replacement2.replace('C', 'g')
replacement4 = replacement3.replace('G', 'c')
print(replacement4.upper())
```

TGACTAGCTAATGCATATCA...

IZLAZ

Mala slova se ne poklapaju sa velikim, pa se zamene ne mešaju.
`upper()` na kraju vraća velika slova.

Traženje greške: ispišite svaki korak

```
replacement1 = my_dna.replace('A', 'T')
print(replacement1)
replacement2 = replacement1.replace('T', 'A')
print(replacement2)
# ... isto za C → G i G → C, uz print posle svakog koraka
```

```
ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT
TCTGTTTCGTTTTTCGTTTTTGTTTTTGCTTTCTTTCTTTTTTTTTTCGTTGCGTTCTT
ACAGAACGAAAACGAAAAGAAAAAGCAAACAAACAAAAAAACGAAGCGAACAA
AGAGAAGGAAAAGGAAAAGAAAAAGGAAAGAAAGAAAAAAAGGAAGGGAAGAA
ACACAACCAAAACCAAAACAAAAACCAACAAACAAAAAAACCAACCCAACAA
```

original, za poređenje

A → T: ispravno

T → A: vraća i T nastala od A

C → G

G → C: vraća i G nastala od C

Dva ispravna rešenja

Privremena slova

```
replacement1 = my_dna.replace('A', 'H')
replacement2 = replacement1.replace('T', 'J')
replacement3 = replacement2.replace('C', 'K')
replacement4 = replacement3.replace('G', 'L')
replacement5 = replacement4.replace('H', 'T')
replacement6 = replacement5.replace('J', 'A')
replacement7 = replacement6.replace('K', 'G')
replacement8 = replacement7.replace('L', 'C')
print(replacement8)
```

Mala slova, korak po korak

```
tCTGtTCGtTTtCGTtTtGTtTTTGCTtTCtTtCtT...
tCaGtaCGtaatCGatatGataaaGCataCtatCta...
tgaGtagGtaatgGatatGataaaGgatagtatgta...
tgactagctaatgcatatcataaacgatagtatgta...
TGACTAGCTAATGCATATCATAAACGATAGTATGTA...
```

Zamena razlikuje velika i mala slova, pa se već zamenjene baze više ne diraju. Na kraju `upper()` vraća velika slova.

EcoRI seče između G i AATTC

1	2	3	4	5
012345678901234567890123456789012345678901234				
ACTGATCGATTACGTATAGTAG	AATTC	CTATCATACATATATATCGATGCGTTTCAT		

Rez je posle G na poziciji 21: fragment 1 su pozicije 0–21 (22 baze), fragment 2 je ostatak (33 baze).

```
my_dna = "ACTGATCGATTACGTATAGTAGAATTCTATCATACATATATCGATGCGTTTCAT"
frag1_length = my_dna.find("GAATTC") + 1
frag2_length = len(my_dna) - frag1_length
print("length of fragment one is " + str(frag1_length))
print("length of fragment two is " + str(frag2_length))
```

IZLAZ

```
length of fragment one is 22
length of fragment two is 33
```

Ako mesta nema, find vraća -1, pa program prijavi fragment dužine 0. Kako to sprečiti, vidimo u poglavlju 6.

Prvi pokušaj: greška za jedan

Pogrešno

```
exon1 = my_dna[1:63]
exon2 = my_dna[91:10000]
print(exon1 + exon2)
```

TCGATCGATCGATCGACTGACTAG...

Rezultat počinje drugom bazom: svi brojevi su za jedan preveliki.

Ispravno

```
exon1 = my_dna[0:63]
exon2 = my_dna[90:10000]
print(exon1 + exon2)
```

ATCGATCGATCGATCGACTGACTAG...

Baza broj n ima indeks $n - 1$, a kraj isečka nije uključen: baze 1–63 su **[0:63]**.

Postavka: egzon 1 ide od početka do 63. baze, egzon 2 od 91. baze do kraja; između je intron.

Koliki deo sekvence je kodirajući?

```
exon1 = my_dna[0:63]
exon2 = my_dna[90:10000]
coding_length = len(exon1 + exon2)
total_length = len(my_dna)
print(coding_length / total_length)
print(100 * coding_length / total_length)
```

IZLAZ

```
0.7804878048780488
78.04878048780488
```

- Dužinu dobijamo iz već napisanog koda: `len(exon1 + exon2)`
- Množenje je `*`, a ne `x`
- Previše decimala sređujemo kasnije (round, pogl. 5)

Intron malim slovima, na tri načina

```
exon1 = my_dna[0:63]
intron = my_dna[63:90]
exon2 = my_dna[90:10000]
print(exon1 + intron.lower() + exon2)
```

```
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCG
AtcgatcgatcgatcgatcgatcatgctATCATCGATCGATATCGATGCATCGACTACTAT
```

```
# ili: intron odmah pretvorimo u mala slova
intron = my_dna[63:90].lower()
# ili: sve u jednoj naredbi, bez promenljivih
print(my_dna[0:63] + my_dna[63:90].lower() + my_dna[90:10000])
```

Birajte zapis čija je namera najjasnija, a ne najkraći.

Rezime i vežbe

Šta smo naučili

- Naredbe, funkcije i argumenti
- Komentari i čitanje poruka o greškama
- Promenljive i tipovi: str i int
- Razlika između funkcija i metoda
- Spajanje, dužina, velika i mala slova
- Zamena, isecanje, brojanje i traženje

Vežbe

1. AT sadržaj zadate DNK sekvence
2. Komplement DNK sekvence
3. Dužine fragmenata posle sečenja enzimom EcoRI (G*AATTC)
4. Introni, 1. deo: ispišite samo kodirajuće regione (egzon 1 do 63. baze, egzon 2 od 91. baze do kraja)
5. Introni, 2. deo: koji procenat sekvence je kodirajući?
6. Introni, 3. deo: egzoni velikim, intron malim slovima